Request-Lifecycle Profiling for Diagnosing LLM Serving Latency

Anonymous Authors
Anonymous Institution
anonymous@example.com

ABSTRACT

Aggregate latency metrics reveal that an LLM request is slow, but not whether the delay arose in admission, prefill, decoding, response delivery, or cleanup. We investigate a request-lifecycle profiler that joins ordered events across these stages, rejects incomplete traces, and applies explicit attribution rules to complete spans. Checked-in probes against an existing NPU server provide three controlled workload contrasts. An injected slow-reader delay is assigned to streaming for all four measured requests; decode-heavy and prompt-heavy/low-output contrasts are assigned to decode for four of four and prefill for eight of eight measured requests, respectively. These repeated requests validate scenario correspondence, not independent estimates of diagnosis accuracy. In a separate concurrency sweep, the profiler localizes two 12.2 s-class tails to the coarse runtime prefill span, narrowing the next experiment without identifying a scheduler, KV-cache, graph, or kernel root cause. Queue, KV-pressure, and cleanup ground truth, controlled online runs, and broad overhead measurements remain missing. The current evidence therefore supports the diagnostic method and a focused research agenda, but not a submission-ready evaluation. Separately, a real-online matched worker-failure study found resource pathology in 1/10 pairs and matched latency pathology in 0/10, below its preregistered 8/10 gate; it therefore rejected the proposed reconciliation treatment for that fixed scenario without rejecting the broader research topic.

KEYWORDS

LLM serving, latency diagnosis, lifecycle tracing, observability

1 INTRODUCTION

Modern LLM serving systems schedule token-by-token execution, manage a shared KV cache, and stream results to clients. Systems such as Orca, vLLM, and SGLang optimize these mechanisms to improve latency or throughput [3, 5, 6]. Their aggregate metrics, however, collapse distinct causes into the same symptom. A high time to first token (TTFT) might reflect admission delay, prefill execution, cache management, or a graph transition. High end-to-end latency with normal TTFT might instead be caused by decoding or by a slow consumer. Choosing an optimization from the aggregate number alone can target the wrong stage.

Distributed tracing links work performed for one request across components [4]. General latency profilers can also identify functions associated with tail events [1]. LLM serving adds a semantic boundary that these abstractions do not by themselves supply: a useful diagnosis must distinguish ordered serving stages and must say when missing events make that diagnosis unsafe. The research question in this draft is therefore: *can ordered request-lifecycle events provide an auditable stage diagnosis that is more actionable than aggregate request timers?*

Our approach assigns each request a sequence of stage events from receipt through cleanup. It derives spans only when both endpoints are present, reports schema completeness, and applies explicit causal rules to complete spans. The rules are intentionally simple: the largest valid span supplies the default stage diagnosis, while stage-specific metadata may refine it (for example, a KV-pressure marker can refine a queue or prefill diagnosis). This separation between observations, completeness, and rules makes a conclusion inspectable and falsifiable.

This draft makes three contributions, each with a deliberately narrow evidence boundary:

- a request-lifecycle schema and completeness contract spanning tokenization, queueing, prefill, decode, streaming, and cleanup;
- deterministic attribution rules and a controlled-fault audit that records both supported scenarios and missing online ground truth; and
- an empirical case showing that a severe latency tail can be localized to a coarse prefill interval, changing the next instrumentation step without overclaiming a substage root cause.

The evidence is not yet sufficient for a conference submission. All current NPU measurements are probes of an already-running server rather than repository-controlled online experiments, and several central baselines and fault classes remain absent. We retain this limitation because it determines the experiments that must be run next.

1.1 Seven-Question Research Contract

(1) Problem. Can identity-bound lifecycle events attribute a slow LLM request to an actionable serving stage without silently filling missing spans? **(2) Importance.** The same TTFT or tail-latency symptom can imply conflicting scheduler, cache, graph, kernel, or delivery interventions. **(3) Common hidden assumption and related-work boundary.** Aggregate timers, raw application logs, generic distributed tracing, and general latency profilers assume that correlation or function association is sufficient; this work tests whether serving-stage ordering, completeness, and controlled interventions add auditable causal value. **(4) Mechanism hypothesis.** Request-, process-, generation-, and stage-bound receipts plus fail-closed span construction can localize the responsible stage, while matched interventions and stage-specific metadata can distinguish causes within an otherwise identical coarse span. **(5) Feasibility.** Existing-server probes already exercise streaming, decode, and prefill correspondence and expose a 12.2 s-class prefill-localized counterexample, but they lack controlled server custody and several internal boundaries. **(6) Matched evaluation.** Compare the profiler with raw logs, aggregate/simple timers, rules-disabled spans, generic tracing where available, and timed

manual diagnosis under identical requests and controlled single-fault injections. The oracle is the injected stage and fault identity; primary gates are trace completeness, attribution accuracy, unsafe-attribution rate, diagnosis time, and end-to-end overhead with CPU, host memory, and NPU memory disclosed. **(7) Citable takeaway.** A positive result requires repeated blind attribution that improves accuracy or diagnosis time over the strongest baseline within the overhead gate; otherwise the citable result is the bounded localization capability and a no-go for causal attribution.

The study stops when an observer cannot be joined to the same request and process identity, the intervention changes more than one preregistered stage, any required endpoint is absent, or the strongest baseline is not run under the same trace. Missing queue, KV-pressure, cleanup, graph, or kernel ground truth remains "evidence required" rather than a zero. A result is labeled *existing-server probe*, *simulation/model*, or *derived artifact* unless the harness owns the complete endpoint, process, request, workload, and hardware receipts needed for real-online evidence.

1.2 Ascend Architecture and Portability Boundary

The architecture fact is that Ascend serving can place scheduler dispatch, worker execution, graph capture/replay, KV allocation, and device prefill inside one coarse prefill-visible interval. This invalidates the assumption that a large TTFT or prefill span names its root cause. The real counterexample is the checked-in existing-server trace with two 12.2 s-class events: the profiler localizes them to prefill but cannot distinguish handoff, KV, graph, or device execution. The proposed native observation contract is to bind timestamps and state receipts on both sides of those accelerator-runtime boundaries. Because that chain and a repository-controlled graph-mode rerun are absent, the current NPU contribution is architecture validation and instrumentation adaptation, not a native Ascend optimization. Its predictive boundary is the recorded Qwen2.5-7B existing-server carrier and measured workloads; the generalization class is request-oriented serving runtimes whose control, state, and device events can share stable identities.

2 BACKGROUND AND MOTIVATION

2.1 Why aggregate timers are ambiguous

TTFT covers work before the first visible token, while time per output token (TPOT) summarizes generation after that point. Neither metric uniquely maps to a serving mechanism. Queueing, tokenization, prefill, and runtime transitions can all inflate TTFT; decode execution and client backpressure can both inflate end-to-end latency. A timer can rank slow requests but cannot, without further structure, distinguish these explanations.

This ambiguity matters because the remedies conflict. Queueing suggests an admission or scheduling experiment. KV pressure suggests cache occupancy and allocation instrumentation. A slow stream reader suggests isolating response delivery, not tuning accelerator kernels. The profiler is useful only if it changes this next decision and if its uncertainty is explicit.

3 PROFILER DESIGN

3.1 Ordered lifecycle events

An event contains a request identifier, stage, monotonic timestamp, observer, and optional metadata. The sequence records receipt, tokenization, queue entry, scheduling, prefill completion, first token, decode completion, stream completion, and cleanup. Its adjacent semantic endpoints define six spans.

Proxy events describe HTTP send, first chunk, stream completion, and client cleanup. Runtime hooks describe internal boundaries. A proxy diagnosis is a client-visible hypothesis; it becomes an internal diagnosis only when runtime events support the same boundary.

3.2 Completeness before attribution

For every request, the analyzer enumerates missing stages and spans. It never fabricates a duration from a single endpoint. Completeness is part of the diagnosis so an absent event cannot silently move blame elsewhere.

3.3 Explicit causal rules

The default rule selects the largest complete span above a duration threshold. Optional metadata may refine that label: a queue or prefill span with an explicit KV-pressure marker is reported as KV pressure. The output names the span, duration, and reason. Rules can be disabled to recover a timer-only baseline.

The rules do not infer a kernel, graph, or scheduler cause from a coarse prefill span. Such a conclusion requires timestamps and counters on both sides of the candidate substage.

4 EVALUATION METHOD

4.1 Questions

We ask whether spans correspond to controlled, externally known stage changes, and how far current events can localize a non-obvious tail. A complete evaluation must later compare causal rules with raw logs, simple timers, rules-disabled reports, and timed manual diagnosis.

4.2 Evidence provenance

The checked-in NPU results are *existing-server probes*: the repository sent requests to a managed Qwen2.5-7B endpoint on NPU6 but did not control the serving process. Derived diagnoses therefore support observations about these requests, not clean comparisons or online accuracy. The manifests also do not establish graph mode, so these are not formal graph-mode results.

Slow streaming is controlled by adding an 80 ms delay while the client reads each chunk. Decode and prefill contrasts use the same structured-agent prompt shape with 128 and four requested output tokens, respectively. The concurrency sweep uses the same request shape at measured concurrency one, two, and three. Repeated requests within a scenario share configuration and server state; they are observations of that scenario, not independent lifecycle experiments.

5 CURRENT RESULTS

5.1 Controlled correspondences and coverage gaps

Table 1 gives the complete coverage audit. Under the injected slow-reader delay, the client-visible diagnosis is streaming for all four measured requests, while runtime hooks record five complete request chains. The decode-heavy contrast is diagnosed as decode for four of four requests, again with five complete runtime chains. The prompt-heavy/low-output contrast is diagnosed as prefill for eight of eight requests, with nine complete runtime chains. These correspondences show that the event ordering can separate the three stages in these checked-in probes. They are not an estimate of population accuracy: there are only three scenario configurations, and their repeated requests are not independent.

The missing rows are scientifically important. No checked-in repository-controlled run provides queue-delay ground truth, KV allocation or pressure ground truth, or an injected cleanup stall. The simple largest-span timer also agrees with the causal-rule output on the three available scenarios, so these cases do not yet show an advantage for causal metadata. A stronger evaluation needs a case—most plausibly KV pressure—where the same coarse timer span admits competing causes and the metadata rule disambiguates them.

5.2 A non-obvious concurrency tail

The concurrency sweep contains 27 measured requests and 30 complete runtime chains including warmups. Table 2 shows a sharp, non-monotonic tail: at concurrency two, the runtime prefill p95 is 12,262.0 ms and two chains exceed one second; concurrency one and three have prefill p95 values of 150.3 and 106.9 ms. Prompt tokens (2,335), requested output tokens (eight), and median cached-token ratio (0.987) are the same across the three blocks.

Table 2: Post-hoc analysis of existing-server runtime traces. The two 12.2 s-class events are inside `scheduled`→ `prefill_done`; the table does not identify a prefill substage cause.

Conc.	Chains	Prompt/output	Prefill median/p95 (ms)	> 1 s
1	9	2335/8	64.8/150.3	0
2	9	2335/8	98.7/12262.0	2
3	9	2335/8	66.4/106.9	0

This is the strongest non-obvious result, but it is localization rather than root cause. Current metadata cannot distinguish scheduler-to-worker handoff, KV allocation/cache pressure, graph capture or batch-shape transition, and device prefill execution. The profiler nevertheless changes the next step: instead of rerunning a broad latency sweep, the frozen follow-up must record scheduler dispatch and worker-start timestamps, KV allocation duration and free-block counts, graph key/capture duration and batch shapes, and device prefill start/end timing. Only a controlled graph-mode rerun with those fields could promote one candidate cause.

5.3 Overhead evidence

The repository contains two small paired probes, but neither supports a broad overhead claim. A client-side pair changes TTFT p95 by +4.07 ms when proxy event construction is enabled; this is client-probe overhead, not runtime-hook overhead. A separate 12-request runtime-hook pair changes TTFT p95 by -2.14 ms and latency p95 by +0.93 ms. Its size and workload do not support TPOT, CPU, host-memory, NPU-HBM, or general-workload conclusions. We therefore exclude overhead from the headline results.

5.4 Issue 19 as a scoped decision gate

A separate real-online study fixed a Qwen2.5-14B workload, request order, single-device runtime configuration, pending-transfer boundary, and worker-exit-last fault before observing results. All ten matched fault/control pairs were evidence-valid. One pair retained an orphan lease and run-owned mmap, while the other nine conserved resources; no pair met the matched latency-pathology condition. Resource and latency pathology counts of 1/10 and 0/10 were both below the preregistered 8/10 gate, so the decision was `NO_GO` and the proposed lifecycle reconciliation remained disabled.

This decision applies only to that fixed worker-exit/pending-transfer scenario. It does not establish that other disconnect, timeout, receipt-loss, multi-Worker, or multi-rank scenarios are safe, and it does not reject Request Lifecycle as a research topic. Complete evidence and longest-stage localization are not by themselves causal-root-cause proof.

6 LIMITATIONS AND REQUIRED EVIDENCE

The current evaluation has four submission blockers. First, the server process was not launched by the benchmark harness, so configuration custody and graph mode are not established by the manifests. Second, queue, KV-pressure, and cleanup ground truth are missing. Third, raw-log, rules-disabled, and timed manual baselines have not been measured. Fourth, the overhead pair is too small to characterize TPOT or memory costs.

These are evidence blockers, not writing tasks. The project should remain a research draft until a frozen, repository-controlled protocol produces graph-mode online runs for the missing fault classes and baselines. A hardware run is warranted only after the exact protocol, admission checks, device availability across NPU0–6, and output manifests are fixed. NPU7 is outside scope. Until those conditions hold, further claim polishing would only hide the evaluation gap.

7 RELATED WORK

LLM serving systems focus on scheduling, batching, cache management, and program execution. Orca's iteration-level scheduling, vLLM's paged KV-cache management, and SGLang's structured execution runtime expose why a request can accumulate latency in different stages [3, 5, 6]. Clockwork studies predictable model-serving latency under controlled execution [2]. Our work does not propose a competing scheduler or cache policy; it asks how to determine which of these mechanisms should be examined for a particular request.

Distributed tracing systems such as Dapper correlate work across services while controlling instrumentation cost [4]. LDB uses event tags to find functions associated with tail latency in

Table 1: Coverage audit over checked-in NPU6 artifacts. Available measurements are probes of an existing server, and the table is derived from those probes. The concurrency tail is an observation, not a controlled-fault row.

Scenario	Control or observation	Lifecycle result	Evidence boundary
Slow client reads	Injected client delay	streaming, 4/4 requests	Existing-server probe; 5/5 complete runtime chains.
Decode-heavy output	Output-length contrast	decode, 4/4 requests	Existing-server probe; 5/5 complete runtime chains.
Prompt-heavy, low output	Prompt/output contrast	prefill, 8/8 requests	Existing-server probe; 9/9 complete runtime chains.
Concurrency-2 tail	Observed anomaly	Coarse prefill span	Existing-server probe; substage cause unresolved. This row is an observation, not a controlled fault.
Queue pressure / KV pressure / Cleanup stall	Not collected	Not evaluated	Requires three repository-controlled online runs with known ground truth.

multithreaded applications [1]. Our profiler adopts their request-centric viewpoint and adds serving stages, completeness rules, and controlled-fault correspondence. Whether these additions improve diagnosis time or accuracy over general tracing remains an evaluation question, not a result of this artifact.

8 CONCLUSION

Ordered lifecycle events can separate streaming, decode, and prefill behavior in three controlled existing-server probes, and can narrow a severe concurrency-sensitive tail to a coarse prefill interval. The latter result is useful precisely because it stops short of naming an unsupported KV, graph, scheduler, or kernel cause and instead defines the next measurements. Missing online ground truth and baselines mean the paper is a research draft, not a submission candidate. Issue 19 likewise shows that complete real-fault evidence can support a scoped `NO_GO` and stop an unsupported treatment without becoming a topic-level negative result.

GENERATIVE AI DISCLOSURE

Generative AI tools assisted with code and prose drafting. The authors reviewed the resulting changes, checked claims against repository artifacts, and remain responsible for the scientific content and any errors.

REFERENCES

[1] Inho Cho, Seo Jin Park, Ahmed Saeed, Mohammad Alizadeh, and Adam Belay. 2024. LDB: An Efficient Latency Profiling Tool for Multithreaded Applications. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 1497–1510.

[2] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 443–462.

[3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626.

[4] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Technical Report dapper-2010-1. Google.

[5] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 521–538.

[6] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Huang, Jeff Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *38th Annual Conference on Neural Information Processing Systems*.